

Background Research done by Alondra Trejo

When we first started working on this project one of the biggest hurdles for me was the UI design. While I enjoy creating interactive designs with animations and pretty colors, that just would not have worked well for users that were doctors. Most doctors do not like wasting time, nor do they like when they are actively being distracted. Therefore Patrick and I began hashing out our ideas in canva, once we had a basic layout we began to search what colors would exude calmness or be less straining in the eye, and although that goes against a design rule, we figured doctors were already busy enough to really care for how a website looked, if anything practicality is what matters the most to them. Of course our hearts as designers would not allow us to create something simple and ugly and leave it as is so we compromise. Once we had the mockups ready, I asked my sister, a scribe, if she could show the design to a doctor. With their feedback we were able to gather that some of the pages were a bit too tricky to understand, and that the colors were too vibrant. Once more we took it back to the mockups, and once again asked for feedback, when it was more or so approved we began our implementation. All of this research was important, as the user is the most important when it comes to great design, specially when that user is not the typical user looking for interactive designs.

Now local LLMS was a different story, my research was quite extensive, from finding which could work with my computer architecture, and which could not, to figuring which would give us the best result and the less wait time. I figured that each had it's set backs, while the bigger models provided a more detailed summary and performed redaction all at once, they traded it for time. Time that we could not give, as a major part of our project was cutting time doctors would need to perform the same services themselves. Once it was decided that models would only summarize, it became a free for all, I began testing PHI4-mini, mistral, medalpaca and others. The models that at first had been discarded proved to be more efficient. Testing this models also took quite some time, hooking them up correctly, and booting them up so it would work correctly with our project became quite inefficient but quite worth it.

One of my final tasks and research was figuring out an authentication service that would work with our project and cause the least amount of time to wire. CAS became the first service I tried, I did not know much of it, so wiring it proved became quite insanity, and since they had multiple versions even AI at times became stumped in how to help with the errors being thrown when trying to build. When asking AI became inefficient I went old school, I looked at all their

documentation, every little piece I believed would be helpful, but their UI really proved difficult to follow. Although I managed to wire it up, and get it to integrate into our project, at the last minute I realized that regretfully enough password reset no longer was compatible with mongoDB. Having no other choice I had to unwire and untangle CAS from the project. While I thought my time was a bit wasted I at least learned my lesson to check if certain databased are compatible before wiring a service. With that in mind I continue searching and stumbled upon auth0, a service compatible with mongoDB, and did not require docker. Although it proved difficult to wire at first, their UI not helpful either, it was significantly faster than CAS. Although it is a paying service, their free trial was for a month so enough for the projects completion. However a big set back with this service is that if you want to inject your own code their service for password reset is no longer valid. Frustrating but something that needed to be sacrificed to meet requirements.